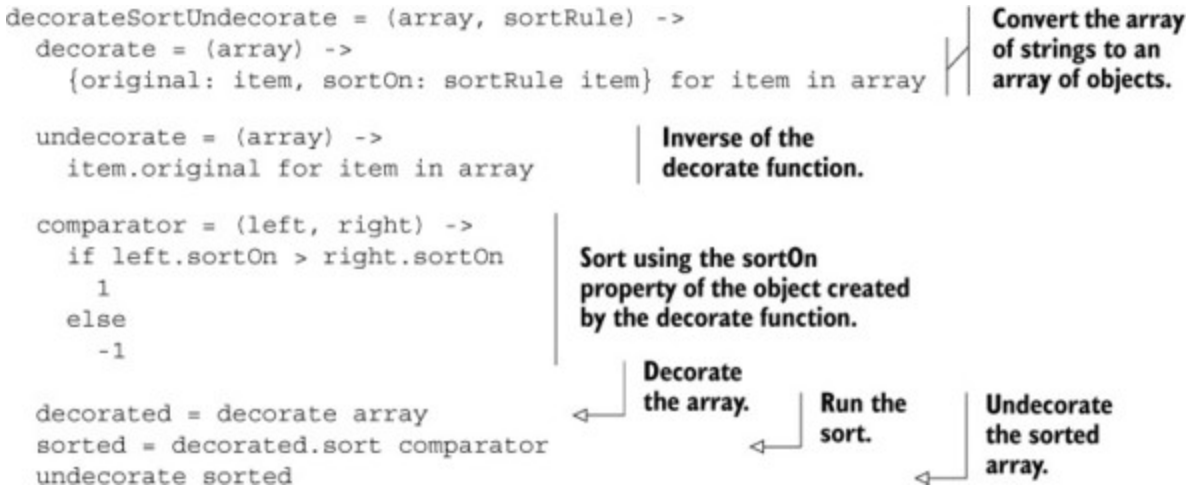


The `decorate` function invoked with an array of strings containing competitor names returns an object with a `sortOn` property. This property contains the string used to compare the competitor to other competitors:

```
lastName = (s) -> s.split(/\s+/g)[1].replace /,/, ''
sortRule = (name) -> lastName name
decorate = (array) ->
  {original: item, sortOn: sortRule item} for item in array

decorate ['0011: Engholm, Ciara']
# [{original: '0011: Engholm, Ciara', sortOn: 'Engholm'}]
```

The `undecorate` function does the reverse. It takes an object created by `decorate` and returns the original string:

```
undecorate = (array) ->
  item.original for item in array

undecorate [{original: '0011: Engholm, Ciara', sortOn: 'Engholm'}]
# ['0011: Engholm, Ciara']
```

Now you can sort the array much faster. An example comparison between the performance with and without decorate-sort-decorate appears in table 9.1. Although actual results will vary significantly across different environments and computers, the decorate-sort-undecorate version is always substantially faster because it does less work.

Table 9.1. Performance improvement of decorate-sort-undecorate